AWS DEPLOYMENT READINESS AUDIT

Dynasties Platform — production gate sprint review

Hostile-but-fair, read-only assessment of whether `github.com/lifecoacher/dynasties-platform` `main` at SHA `62d3b47` can be deployed to AWS for real-world testing. Independent verification of the seven hardening claims from the latest sprint, an end-to-end AWS readiness review, and a final ship/no-ship judgment.

Repository	`lifecoacher/dynasties-platform`
Branch	`main`
Local HEAD	`62d3b476ee5c17cf1b16fa0a3a16531cac220af7`
origin/main HEAD	`62d3b476ee5c17cf1b16fa0a3a16531cac220af7`
Working tree	clean — no modifications
Sprint commit	`62d3b47` "Improve security and stability by fixing migration issues and refining error handling" — landed since prior audit
Audit date	May 4, 2026
Mode	READ-ONLY — no fixes applied, no commands altered remote state

1 · Executive verdict

DEPLOYABLE WITH CONDITIONS — NOT YET READY FOR REAL-USER PILOT

The sprint was real. Most of the seven claimed fixes are independently verifiable in source: the migration chain is now `0000→0005` with unique tags and a coherent journal, the orphaned `0003_*` file has been renamed to `0005_recommendation_hardening.sql` and registered. Vitest is wired into the root `test` script with a real `vitest.config.ts`. Helmet now ships with a strict CSP. CORS production validation is enforced at the Zod schema level. Pino is actually imported and used by the request logger and error handler with redaction and tenant context. Dockerfiles pin pnpm@10.26.1. The Stripe webhook handler distinguishes 4xx signature failures from 5xx internal exceptions. Process-level error handlers now drain and exit. `scripts/run-migrations.sh` is genuinely used by both CI and deploy.

But the codebase is still tied to Replit in production-critical paths in ways that will break a real AWS deployment the moment users try to pay you. The Stripe webhook URL and every checkout return URL is constructed from `process.env.REPLIT_DOMAINS`; on AWS that is undefined and you will register webhooks at `https://undefined/...`. The Terraform ECS task definition omits `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `CLERK_WEBHOOK_SECRET`, and the QuickBooks credentials entirely — without them, billing, webhooks, and accounting silently degrade or fall back to demo mode. The deploy workflow runs Drizzle migrations from a GitHub Actions runner against an RDS

instance that lives in a private subnet — which will fail at the network layer the first time you actually try to deploy. The "46 tests passing" claim is misleading: about 26 are E2E specs that require a running API server CI does not start, and the unit suite is small (~15) and partially tautological.

Bottom line: **deploy this to a private internal AWS environment and exercise it yourself, fine. Point real paying users at it without fixing the items below — that is a money-path outage waiting to happen.**

2 · Git sync status

Local HEAD	`62d3b476ee5c17cf1b16fa0a3a16531cac220af7`
origin/main HEAD	`62d3b476ee5c17cf1b16fa0a3a16531cac220af7`
Equality	**EQUAL — no divergence**
Working tree	**CLEAN — no modifications, no staged changes**
Detached HEAD	Yes — checked out the audited SHA in the sandbox clone (read-only, does not affect any other workspace)
Last sprint commit	`62d3b47` "Improve security and stability by fixing migration issues and refining error handling" (24 files changed, 34,356+/56-)
Caveat	Audit can only see GitHub remote and the sandbox clone — your live Replit workspace was not inspected directly

3 · Code review confidence

High. The codebase is syntactically clean (zero `@ts-ignore`, zero `TODO/FIXME/HACK` markers, zero `eslint-disable`), uses a modern monorepo layout with pnpm workspaces, has a non-trivial Terraform definition for VPC/RDS/ECS/ALB/CloudFront, and the patterns are consistent. The sprint shows a senior author addressing real findings methodically. The shape of the code is right. The remaining gaps are deployment-environment integration, not architectural.

4 · AWS readiness level

LEVEL 2 — Can deploy to AWS for internal testing with major caveats

Why not Level 3: the Replit-coupled URL construction and the missing Stripe/Clerk/QB secrets in the ECS task definition are not theoretical — they will produce broken behavior on the first real billing flow. Why not Level 1: containerization, env validation, migrations, IaC, CI typecheck/test/build, and ECS task plumbing all exist and look correct in the parts they do cover. An internal team running a non-billing dry run would likely succeed; a real-user pilot that exercises Stripe checkout would not.

Level 0	Not deployable — basic build/run broken
Level 1	Boots in dev only — would not survive container orchestration
Level 2 ←	**CURRENT** — deploys to AWS for internal testing with major caveats
Level 3	Controlled external pilot with real users
Level 4	Operationally strong for broader production rollout

5 · Verified strengths

✓ Migration chain is now coherent. `lib/db/migrations/_journal.json` registers all six tags `0000_neat_microchip` through `0005_recommendation_hardening`. The previously orphaned file is renamed and registered. Six SQL files match six journal entries.

✓ Vitest test runner is wired up correctly. `vitest.config.ts` exists with `tests/**/*.test.ts` + lib/artifacts globs. Root `package.json` has `"test": "vitest run"`. CI runs `pnpm test` with a real Postgres service container.

✓ Helmet CSP is real. `app.ts` sets `contentSecurityPolicy: { directives: { defaultSrc: ["'none'"], frameAncestors: ["'none'"], connectSrc: ["'self'"] } }`. Production-grade tightness for an API surface.

✓ CORS production validation enforced at schema level. `lib/config/src/env.ts` defines `corsOriginsSchema` with a Zod refine that fails validation if `CORS_ALLOWED_ORIGINS` is empty when `NODE_ENV` is `production` or `staging`. The boot will throw before serving traffic.

✓ JWT secret known-bad-substring guard. `jwtSecretSchema` in `env.ts` rejects secrets containing `dev/change/example/secret/password/placeholder` in production/staging. `docker-compose.yml` default fallback removed: `JWT_SECRET: "${JWT_SECRET}"`.

✓ Pino structured logging genuinely in use. `lib/config/src/logger.ts` exports a pino instance with redaction for `authorization`, `cookie`, `password`, `token`, `secret`, `email`, `stripeCustomerId`, `refreshToken`. `app.ts`, `request-logger.ts`, `error-handler.ts` all import and use it. Per-request child logger with `requestId`, `userId`, `companyId`.

✓ Demo route ordering deterministic. `app.ts` mounts `demoRouter` behind `(isDemoMode || !isProduction)` via static import, before `notFoundHandler` and `globalErrorHandler`. Production with `VITE_DEMO_MODE` unset will log `'Demo routes disabled'` and skip the mount.

✓ Process-level error handlers now graceful. `index.ts` installs `uncaughtException` and `unhandledRejection` handlers that log via pino at `fatal`, close the HTTP server, drain the DB pool, and `process.exit(1)` with a 15s force-exit timeout. ECS will replace the task as intended.

✓ Stripe webhook 400 vs 500 separation. `app.ts` stripe webhook route inspects `error.type === 'StripeSignatureVerificationError'` and returns 400/log warn for signature failures, 500/log error for downstream exceptions. Stripe retry behavior is now correct.

✓ Dockerfiles pin pnpm@10.26.1. Both `Dockerfile` and `Dockerfile.frontend` use `corepack prepare pnpm@10.26.1 --activate`. CI `ci.yml` sets `PNPM_VERSION: '10.26.1'`. Single source of truth respected.

✓ Migration runner used in deploy. `.github/workflows/deploy.yml` invokes `bash scripts/run-migrations.sh` in the `deploy` job (line ~78). `ci.yml` also calls it in the `test` job. No drift.

✓ Gitleaks in CI. `.github/workflows/ci.yml` has a `secret-scan` job using `gitleaks/gitleaks-action@v2` with `fetch-depth: 0`. Gates the `build` job.

✓ TRUST_PROXY is configurable. `app.set('trust proxy', Number(process.env.TRUST_PROXY || "1"))`. Behind ALB with default 1 works; CloudFront → ALB chain can set 2.

✓ Health endpoints are meaningful. `/api/healthz` performs a live `SELECT 1` against Postgres, checks queue consumer count, reports memory. Returns 503 when degraded. `/api/healthz/ready` and `/api/healthz/live` are simpler — perfect for ECS health-check vs. readiness-probe split.

✓ Terraform infrastructure-as-code is non-trivial. `infra/main.tf` (756 lines) defines VPC, public/private subnets across two AZs, NAT gateway, RDS Postgres 16.4 with encrypted storage, ECS Fargate cluster + task + service, ALB with HTTPS listener and HTTP→HTTPS redirect, target group health check, ECR repos, S3 frontend bucket, CloudFront distribution with OAI, SSM SecureString parameters for secrets injection. Real.

✓ No tech debt markers. Zero `TODO/FIXME/HACK/XXX`, zero `@ts-ignore/@ts-expect-error`, zero `eslint-disable` across non-test source. Code hygiene is high.

6 · CRITICAL AWS blockers

These will produce broken behavior on AWS the first time the affected code path runs with real traffic. Fix before any pilot exposed to paying users.

CRITICAL — Stripe webhook URL is hard-coded to `REPLIT_DOMAINS` — registers `https://undefined/...` on AWS

`artifacts/api-server/src/index.ts (line 75)`

Issue	`const webhookBaseUrl = \`https://${process.env.REPLIT_DOMAINS?.split(',')[0]}\`;` — on AWS `REPLIT_DOMAINS` is undefined, so the optional chain returns `undefined` and the URL becomes the literal string `https://undefined/api/stripe/webhook`. `stripeSync.findOrCreateManagedWebhook()` is then called with that URL.
Why on AWS	Stripe webhook registration silently registers a webhook pointing at `undefined`. Stripe events will never reach your ECS service, so subscription state, invoice status, deployment-fee payment status will all drift out of sync. Webhook idempotency tables stay empty. Customers pay and your DB never knows.
Recommended fix	Add a required `PUBLIC_BASE_URL` env var (e.g. `https://app.dynasties.io`) to the Zod env schema, fail boot in production if missing. Replace every `REPLIT_DOMAINS`/`REPLIT_DEV_DOMAIN` reference in `index.ts`, `stripe-billing.ts`, `stripe-connect.ts`, and `app.ts` CORS with `env.PUBLIC_BASE_URL`. Wire it into the Terraform ECS task definition `environment` block.

CRITICAL — ECS task definition is missing Stripe, Clerk, and QuickBooks secrets

`infra/main.tf (lines 386-415, ECS task container_definitions)`

Issue	The Fargate task definition injects only `DATABASE_URL`, `JWT_SECRET`, and `ANTHROPIC_API_KEY` from SSM. `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PUBLISHABLE_KEY`, `CLERK_WEBHOOK_SECRET`, `QB_CLIENT_ID`, `QB_CLIENT_SECRET`, and `QB_MODE` are all declared optional in the env schema and missing from the ECS task entirely.
Why on AWS	On boot the api-server will not have a Stripe key. The fallback in `stripeClient.ts` (`getCredentialsFromReplit`) requires `REPLIT_CONNECTORS_HOSTNAME`, which AWS does not have, so it returns null. Any billing endpoint that touches Stripe throws at runtime. `STRIPE_WEBHOOK_SECRET` is undefined, so the webhook handler in `webhookHandlers.ts` hits the production-throw branch (line 88: `throw new Error("STRIPE_WEBHOOK_SECRET is required in production")`) on the first event Stripe sends — except Stripe will not send any because of the prior issue. `QB_MODE` defaults to demo when unset, so QuickBooks integration runs in demo mode without warning.
Recommended fix	Add SSM SecureString parameters for each missing secret in `infra/main.tf`, then add them to the task definition `secrets` array. Add `QB_MODE = "live"` to the plain `environment` array. Mark them required in the Zod env schema for production. Apply the same treatment to `CLERK_WEBHOOK_SECRET` in the clerk webhook router.

CRITICAL

Migration step in deploy.yml runs from GitHub Actions runner against private-subnet RDS — will fail at network layer

`.github/workflows/deploy.yml (lines 75-80) · infra/main.tf (RDS subnet group)`

Issue	The deploy job runs `bash scripts/run-migrations.sh` on the GitHub-hosted runner with `DATABASE_URL: ${{ secrets.DATABASE_URL }}`. RDS is provisioned with `publicly_accessible = false` in private subnets and a security group that only allows ingress from the ECS task SG (per the IaC pattern). The runner's IP is not in any allowed range.
Why on AWS	First production deploy fails at the `npx drizzle-kit migrate` step with a TCP timeout. ECS service then either (a) tries to start with a schema mismatch and crashes on first query, or (b) the deploy job aborts and you have a half-deployed system.
Recommended fix	Move migrations into a one-off ECS Fargate task launched by the deploy job: `aws ecs run-task --task-definition dynasties-migrate --launch-type FARGATE --network-configuration ...`. The migration task runs inside the VPC with the same RDS access as the api task. Wait for `aws ecs wait tasks-stopped` and check exit code before proceeding to `update-service`. Alternative: run migrations on api-server boot before `app.listen()` — but that races multiple ECS tasks and is operationally worse.

CRITICAL

CORS_ALLOWED_ORIGINS in Terraform points only at the CloudFront default domain — custom domain users will be blocked

`infra/main.tf (line 397) · artifacts/api-server/src/app.ts (CORS config)`

Issue	`CORS_ALLOWED_ORIGINS = "https://${aws_cloudfront_distribution.main.domain_name}"`. The CloudFront default domain is something like `d1234abcd.cloudfront.net`. If you point your Route 53 record at `app.dynasties.io`, the browser's `Origin` header will be `https://app.dynasties.io`, which is not in the allow list, and the api-server will reject every preflight.
Why on AWS	Real users hitting the production domain see a blank page or CORS errors in the console for every API call. Cookies with `SameSite=Lax` won't be sent across origins anyway, so credentialed flows fail twice.
Recommended fix	Set `CORS_ALLOWED_ORIGINS` to a comma-separated list including both the CloudFront default domain and the user-facing custom domain: `"https://app.dynasties.io,https://${aws_cloudfront_distribution.main.domain_name}"`. Drive it from `var.domain_name` in Terraform. Same for any `PUBLIC_BASE_URL`.

7 · AWS risks and caveats

MAJOR `initStripe()` runs `runMigrations` and webhook setup on every boot

`artifacts/api-server/src/index.ts (lines 50-90)`

Issue	Each ECS task boot calls `stripe-replit-sync.runMigrations()` against the same DB and `findOrCreateManagedWebhook` against Stripe. With multi-instance Fargate (`desired_count > 1`) you get a thundering herd of concurrent migration attempts and redundant webhook upsert calls. It also blocks `app.listen()` behind these calls, slowing rolling deploys.
Why on AWS	Migrations should be a separate one-off task (see Critical #3). Webhook setup should be idempotent and out-of-band — a deploy script step, not boot-time.
Recommended fix	Move `initStripe()` out of the boot path. Run schema migrations only in the ECS migration task. Run `findOrCreateManagedWebhook` once per deploy (a separate `register-webhook` ECS task or a manual one-time setup). `app.listen()` should be the very first async work.

MAJOR Stripe checkout success/cancel URLs use `REPLIT_DEV_DOMAIN` first

`artifacts/api-server/src/routes/stripe-billing.ts (lines 223, 266, 288) · stripe-connect.ts (line 76)`

Issue	Every checkout/portal session is built with `const baseUrl = \`https://${process.env.REPLIT_DEV_DOMAIN || process.env.REPLIT_DOMAINS?.split(',')[0]}\`;`. On AWS both env vars are undefined, so customers get redirected to `https://undefined/settings/billing?checkout=success`.
Why on AWS	Customers pay you successfully, then land on a page that does not exist. Even worse, the cancel URL fails the same way, so they cannot back out cleanly.
Recommended fix	Same fix as Critical #1 — replace with a required `PUBLIC_BASE_URL` env var. This is the same root cause; flagging it separately because it lives in product flow code, not infra.

MAJOR

147 `console.*` calls remain in `artifacts/api-server/src`

`artifacts/api-server/src (147 occurrences across many files)`

Issue	The pino migration covered `app.ts`, `request-logger.ts`, and `error-handler.ts`. The rest of the api-server still emits raw `console.log`, `console.warn`, `console.error`. `webhookHandlers.ts` for example uses `console.warn` for the dev-mode signature-skip warning.
Why on AWS	CloudWatch will receive a mix of structured pino JSON and unstructured stderr lines. Log queries based on `requestId`/`companyId` miss anything that goes via `console.*`. Redaction is bypassed — `console.log({ stripeCustomerId })` leaks PII while pino would redact it.
Recommended fix	Sweep the api-server with a codemod: replace every `console.log/warn/error` with a `logger.info/warn/error` call, importing `createLogger` at module scope. Add an ESLint rule `no-console: 'error'` to prevent regression.

MAJOR

`STRIPE_WEBHOOK_SECRET` is optional in env schema but throws in production at runtime

`lib/config/src/env.ts (line 51) · artifacts/api-server/src/webhookHandlers.ts (line 88)`

Issue	`STRIPE_WEBHOOK_SECRET: z.string().optional()` in env schema, but the webhook handler throws `'STRIPE_WEBHOOK_SECRET is required in production'` when missing. The schema does not catch this at boot — only the first incoming Stripe event does, by which point you've already started serving traffic.
Why on AWS	Boot succeeds, the api-server appears healthy, ECS marks the task healthy, ALB routes traffic to it. Then the first Stripe webhook arrives and throws — caught by the 500 branch and returned to Stripe, which retries with backoff. Logs show 500s with no clear root cause unless someone reads the message text.
Recommended fix	Add a Zod refine to `STRIPE_WEBHOOK_SECRET` requiring it when `NODE_ENV === 'production' || NODE_ENV === 'staging'`. Same pattern as `corsOriginsSchema`. Apply to `STRIPE_SECRET_KEY`, `CLERK_WEBHOOK_SECRET`, `QB_CLIENT_ID/SECRET` as well.

MAJOR

ECS service desired count not set in IaC — defaults to 1, no horizontal redundancy

`infra/main.tf (line 531, aws_ecs_service.api)`

Issue	The task definition has 1 CPU / 2 GB. The service block does not visibly set `desired_count` in this snippet. If the default is 1, a single ECS task is the entire production fleet. Rolling deploys with `force-new-deployment` will tear down the only instance and you get downtime windows.
Why on AWS	Single point of failure for the API. Any task replacement, any AZ outage, any healthcheck blip = full outage.
Recommended fix	Set `desired_count = 2` minimum and `deployment_minimum_healthy_percent = 50`, `deployment_maximum_percent = 200`. Add an ECS auto-scaling target tracking CPU. The accounting/queue services may also need the same.

MAJOR

In-memory queue is the default — multi-instance ECS will lose jobs

`lib/queue/src/index.ts · artifacts/api-server/src/extraction-consumer.ts`

Issue	`QUEUE_BACKEND` is optional in env schema. When unset (or set to `local`) the queue uses Node's `EventEmitter` per process. Each ECS task has its own emitter; jobs produced on instance A cannot be consumed by instance B. With `desired_count > 1` you get non-deterministic processing, and the `healthz` queue check will report only one instance's consumer count.
Why on AWS	Background jobs (extraction, recommendations, document generation) silently process only the job-emitting instance's events. Restarting an instance loses any in-flight in-memory jobs.
Recommended fix	Wire `QUEUE_BACKEND=sqs` in the ECS task `environment` block. Provision the SQS queues in `infra/main.tf` (the IAM task role already has SQS permissions claimed in the Zod schema). Verify the SQS adapter implementation in `lib/queue` handles DLQ + visibility timeout correctly under real load.

MEDIUM

Money-path E2E tests require a running server but CI runs them with `continue-on-error: true`

`tests/money-path/money-path.test.ts · .github/workflows/ci.yml (lines 95-100)`

Issue	`tests/money-path/money-path.test.ts` hits `http://localhost:8080/api`. CI starts a Postgres service container but does not start the api-server. The first `fetch()` fails with ECONNREFUSED. CI marks the step yellow but `continue-on-error` swallows the failure. Effectively the 26 money-path tests never validate anything in CI.
Why on AWS	Regressions in billing — invoice creation, charge rules, reconciliation, tenant isolation — will not be caught before merge. The "46 tests passing" claim depends on E2E specs that CI does not actually run.
Recommended fix	Either start the api-server as a CI service container with all required env (Anthropic mocked, Stripe in test mode, etc.) and run the E2E job for real; or split unit tests from E2E and gate merges only on unit tests, marking E2E as a separate workflow that runs against a deployed staging environment.

MEDIUM Env validation tests are partially tautological

`tests/unit/env-validation.test.ts`

Issue	The unit test file re-implements the validation rules inline (`const KNOWN_BAD_SECRETS = [...]` copied from `env.ts`) and then asserts the inline copy works. It does not import `loadEnv` from `@workspace/config` and exercise it. If the real schema in `env.ts` drifts (e.g. somebody removes the refine), the test still passes.
Why on AWS	False confidence. The 'env validation tests' do not validate the env schema; they validate a copy of the rules.
Recommended fix	Refactor to import the real `loadEnv()` with `process.env` stubbed via `vi.stubGlobal` or by passing parsed input directly into the schema. Test failure modes against the production code path.

MEDIUM Stripe credential fallback to Replit Connector still in code

`artifacts/api-server/src/stripeClient.ts (lines 10-50)`

Issue	`getCredentialsFromReplit()` remains as a fallback after env-var lookup. On AWS it returns null and the env path takes over — but the function is dead code in AWS, makes a network call to `REPLIT_CONNECTORS_HOSTNAME` when env vars are absent in any non-prod environment, and adds complexity to the Stripe init path.
Why on AWS	Defensive code that hides a clear separation: AWS gets env vars, Replit gets the connector. Today it works but every future engineer reading the file has to understand both paths.
Recommended fix	Gate `getCredentialsFromReplit` behind `process.env.REPLIT_CONNECTORS_HOSTNAME` and exit early on AWS. Or remove the Replit path entirely once you've fully migrated to AWS.

MEDIUM `frame-ancestors 'none'` CSP blocks Stripe Checkout iframe

`artifacts/api-server/src/app.ts (helmet config) · Stripe Elements / hosted checkout`

Issue	If the frontend embeds Stripe Elements (which uses `js.stripe.com` iframes) or any redirected checkout returns to a page that needs to be embeddable, the CSP `frame-ancestors 'none'` on the API doesn't directly block them — but if the frontend is served behind the same Helmet config in any wrapper, it will. Worth confirming the frontend's nginx config does not inherit this CSP.
Why on AWS	Stripe Elements may render blank or refuse to load. Customer card collection breaks.
Recommended fix	Confirm the frontend's `nginx.conf` sets its own CSP that allows `frame-src https://js.stripe.com https://hooks.stripe.com` and `connect-src https://api.stripe.com`. The API CSP can stay strict.

MEDIUM

`app_user` role assumed by Stripe schema GRANTs but not provisioned in IaC

`artifacts/api-server/src/index.ts (lines 65-70) · infra/main.tf (RDS)`

Issue	Boot runs `GRANT USAGE ON SCHEMA stripe TO app_user; GRANT SELECT ON ALL TABLES IN SCHEMA stripe TO app_user;`. There is no Terraform resource creating an `app_user` Postgres role. RDS by default uses the master user only. The GRANTs will fail silently (caught by `.catch(() => {})`), and the app server will connect as the master user.
Why on AWS	Application connects as DB superuser instead of a least-privilege role. SQL injection blast radius is the entire database, not a constrained schema. Compliance/security review will flag this immediately.
Recommended fix	Either remove the `app_user` GRANT block (since it doesn't run anyway), or add a Terraform `postgresql_role` resource (or a bootstrap migration) that creates the role with limited privileges and have the app connect as `app_user` via a separate SSM-managed connection string.

8 · Re-verified hardening items

Claim	Status	Evidence
Migration chain 0000→0005 with unique IDs and prevId linkage	**VERIFIED**	`_journal.json` registers all 6 tags; SQL files match; idx 0–5 monotonic.
Demo route ordering deterministic, mounted before notFoundHandler	**VERIFIED**	`app.ts` static import + conditional mount, then `notFoundHandler`, then `globalErrorHandler`.
CORS production validation requires non-empty origins	**VERIFIED**	`env.ts corsOriginsSchema` Zod refine fails boot in production/staging when empty.
Helmet CSP truly enabled, not superficial	**VERIFIED**	`defaultSrc 'none'`, `frameAncestors 'none'`, `connectSrc 'self'` set in `app.ts`.
Pino structured logging with redaction	**VERIFIED**	Pino imported and used; redact paths cover authorization, cookie, tokens, email, stripeCustomerId.
deploy.yml uses `scripts/run-migrations.sh`	**VERIFIED**	Both `ci.yml` and `deploy.yml` call it; no inline drift.
46 tests passing	**PARTIAL**	Vitest wired; ~15 unit tests run in CI. ~26 money-path E2E tests are `continue-on-error` and skip due to no live server.
Pnpm version pinning consistent	**VERIFIED**	Both Dockerfiles, ci.yml env, and root `package.json` packageManager all = 10.26.1.
Stripe webhook 4xx/5xx error separation	**VERIFIED**	`app.ts` distinguishes `StripeSignatureVerificationError` (400/warn) from internal (500/error).
TRUST_PROXY env support	**VERIFIED**	`app.set('trust proxy', Number(process.env.TRUST_PROXY || "1"))`.
Gitleaks secret-scan in CI	**VERIFIED**	`secret-scan` job uses `gitleaks/gitleaks-action@v2`; gates the build job.
JWT_SECRET hard-fail with bad-substring guard	**VERIFIED**	`env.ts jwtSecretSchema` rejects dev/change/example/secret/password/placeholder.
Process-level error handler graceful exit	**VERIFIED**	`index.ts` drains HTTP + DB pool, 15s force-exit, then `process.exit(1)`.

Score: 12 of 13 hardening items fully verified. 1 (test count) is partial — the unit suite is real and runs, but the larger E2E suite is silently skipped in CI.

9 · Real-world test readiness by product area

Product area	Readiness	Notes
Auth (Clerk + JWT)	**READY WITH CONDITIONS**	Clerk webhook signature verified (HMAC-SHA256, 5-min window). JWT hard-fails on bad secret. `CLERK_WEBHOOK_SECRET` is missing from ECS task — Clerk events will throw on first delivery.
Frontend app	**READY WITH CONDITIONS**	Builds via Vite + nginx. CloudFront → S3 path is wired in IaC. CORS will block until custom-domain origin is added (Critical #4).
API (Express)	**READY**	Boots, validates env, mounts routes correctly, healthchecks meaningful, structured logging, helmet+CSP, rate limiters present.
Database (RDS Postgres)	**READY WITH CONDITIONS**	Schema migrations valid and ordered; encrypted at rest; private subnet. Migration runner needs to move into ECS task to actually reach the DB (Critical #3).
Billing (Stripe)	**NOT READY**	Webhook URL points at `undefined` on AWS; ECS task missing `STRIPE_*` secrets; checkout success/cancel URLs broken. Money path will not work end-to-end.
Webhooks (Stripe / Clerk)	**NOT READY**	Idempotency, signature verification, 4xx/5xx separation are all correct in code. Production secrets are not injected by IaC, and Stripe webhook URL is not registered correctly.
Background jobs / queues	**READY WITH CONDITIONS**	Code supports SQS via `QUEUE_BACKEND=sqs`; ECS task does not set it; default in-memory queue breaks under multi-instance Fargate.
Logging / monitoring	**READY WITH CONDITIONS**	Pino structured logs to stdout; CloudWatch log group provisioned. 147 `console.*` calls outside the migrated files emit unstructured noise. Prometheus `/api/metrics` exists but no Prometheus scraper provisioned in IaC.
CI / CD	**READY WITH CONDITIONS**	Real typecheck, secret-scan, build, deploy. E2E money-path silently `continue-on-error`. No staging environment in workflow — main goes straight to production.
Deployment scripts	**READY WITH CONDITIONS**	`scripts/run-migrations.sh` shared between CI and deploy. Migration step in deploy.yml hits private RDS from public runner — will fail.
Secrets / env handling	**READY WITH CONDITIONS**	Zod schema enforces JWT/CORS/format rules. SSM SecureString integration in IaC for the secrets that are wired (DB, JWT, Anthropic). Stripe/Clerk/QB secrets are not wired.

10 · Minimum next steps before AWS test

1. Replace Replit URL coupling. Add `PUBLIC_BASE_URL` to the env schema (required in production). Replace every `REPLIT_DOMAINS`/`REPLIT_DEV_DOMAIN` reference in `index.ts`, `stripe-billing.ts`, `stripe-connect.ts`.
2. Add missing secrets to Terraform ECS task. SSM SecureString + ECS `secrets` entries for `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `CLERK_WEBHOOK_SECRET`, `QB_CLIENT_ID`, `QB_CLIENT_SECRET`. Add `QB_MODE=live` and `QUEUE_BACKEND=sqs` to the plain environment block.
3. Move migrations into a Fargate task. Define a one-off `dynasties-migrate` task; have `deploy.yml` run it via `aws ecs run-task` instead of running `scripts/run-migrations.sh` on the runner.
4. Fix CORS allow-list in IaC. Drive `CORS_ALLOWED_ORIGINS` from `var.domain_name`; include both the custom domain and CloudFront default.
5. Make Stripe/Clerk webhook secrets required at boot. Refine the Zod schema so production fails to start if any money-path secret is missing — fail fast, not on first event.
6. Set ECS desired_count ≥ 2. Add `deployment_minimum_healthy_percent = 50` and a basic CPU target tracker.
7. Provision SQS queues in IaC and set `QUEUE_BACKEND=sqs`. Verify the SQS adapter handles DLQ and visibility timeout correctly.
8. Make CI E2E real or remove the claim. Either start the api-server in CI (with mocked external services) and run money-path E2E for real, or move E2E to a post-deploy staging job. Drop `continue-on-error`.
9. Sweep remaining 147 `console.*` calls with a codemod and add ESLint `no-console: 'error'`.
10. Provision `app_user` Postgres role or remove the GRANT block. Connect the app as a least-privilege role.

Estimated cost: 2-3 engineer-days for a senior with AWS + TypeScript fluency. The first three items are roughly half the work and the most important; the rest can land iteratively.

11 · Final recommendation

PUSH TO AWS ONLY AFTER FIXING LISTED BLOCKERS

Specifically:

- Safe to deploy for internal AWS testing now — only if you accept that billing and webhook flows will be broken until you wire the secrets and replace the Replit URL coupling. An internal team running a non-billing smoke test (auth, dashboards, document ingestion, recommendations) would likely succeed.
- NOT safe for limited real-user pilot now. The first user who clicks 'Subscribe' will hit a checkout flow that returns to `https://undefined/`, no Stripe webhook will be registered correctly, and their subscription state will diverge from Stripe's reality. This is a money-path failure, not a polish issue.
- After the four CRITICAL items are fixed (Replit URLs, ECS secrets, migration network access, CORS allow-list) and the four MAJOR items are addressed (boot-time Stripe init, checkout URLs, webhook-secret schema enforcement, ECS `desired_count`) — Level 3 is a realistic two-day target.

Audit conducted in read-only mode. No commands altered remote state. Sandbox clone is at detached HEAD `62d3b47`, working tree clean. The user's Replit workspace was not inspected directly — verification limited to

`origin/main` as published on GitHub.